

Atomic-Connectivity Visualization Adapter Specification

State and transition graph views, frame-local periodic geometry, and 2-D/3-D wrappers

mdstats

Contents

Purpose and status	1
Motive	1
Normative ownership	1
AI context summary	2
Atomic-connectivity adapter	2
Public state adapter	2
Required input consistency	2
Atomic node attributes	2
Atomic edge attributes	3
Canonical versus frame-consistent image shifts	3
The problem	3
Reconstruction algorithm	3
Atomic transition adapter	4
Public definition	4
Transition graph construction	4
Transition keys and attributes	4
Atomic convenience plotting functions	5
Real-structure system-integration fixture	5
Adapter metadata schemas	5
Input constraints	6
Error and warning policy	6
Edge cases and cautions	6
Testing requirements	6
Public exports	6
Future compatibility	7
Interactive 3-D wrappers	7

Purpose and status

This document is the normative specification for `mdstats.plotting.atomic_connectivity_graph`. The state/transition adapters and 2-D wrappers were introduced in `mdstats` 0.10.0; the periodic-aware 2-D path and interactive 3-D wrappers are implemented in 0.11.0.

The adapter converts authoritative `AtomicConnectivityState` and `AtomicConnectivityResult` objects into generic decorated graph views. It also constructs diagnostic union graphs for connectivity transitions and supplies convenience wrappers for the 2-D and 3-D renderers.

Motive

Atomic connectivity is the first real scientific graph consumed by the visualization system. The adapter must preserve atom and edge identities, expose useful metadata, and reconcile canonical periodic graph shifts with the wrapped coordinates of the selected frame.

Normative ownership

This specification owns:

- atomic-state graph-view conversion;
- atomic-transition union graph conversion;

- adapter metadata schemas;
- frame-local display-shift reconstruction;
- atomic and transition 2-D and 3-D convenience wrappers;
- the Na-LTA integration fixture.

It does not own connectivity construction, cutoff selection, state cataloging, generic filtering, styling, layout, or rendering.

AI context summary

- The connectivity state is authoritative; the adapter never decides whether an edge exists.
- Node keys are canonical atom indices.
- State edge keys remain AtomicEdgeKey objects.
- Canonical graph shifts and frame-local display shifts are both retained.
- Display shifts are reconstructed without changing graph winding.
- Transition views are union diagnostic graphs with unchanged, added, and removed edge classes.
- The coordinate frame may be selected from before or after a transition.
- Convenience wrappers use adapter-specific default styles but forward all generic renderer controls.

Atomic-connectivity adapter

Public state adapter

```
def graph_view_from_atomic_connectivity(
    collection: AtomisticFrameCollection,
    connectivity: AtomicConnectivityState |
        AtomicConnectivityResult,
    *,
    frame_index: int,
) -> DecoratedGraphView:
    ...
```

When connectivity is an AtomicConnectivityResult, frame_index must be among the analyzed collection-frame indices and the corresponding state is selected.

When connectivity is an AtomicConnectivityState, frame_index supplies physical coordinates for that state.

Required input consistency

The adapter must verify:

- frame_index is a valid collection position;
- every active atom index exists in the collection;
- active atomic numbers match the collection species;
- the state PBC flags match the selected frame;
- the selected cell and positions are finite;
- the selected frame contains the same fixed atom identity ordering;
- the state digest and arrays are internally valid through the state constructor;
- a result maps the requested frame to exactly one state.

A mismatch is an adapter error. The adapter must not silently guess a frame or remap atom identities.

Atomic node attributes

The adapter must provide at least:

atom_index	int
atomic_number	int
symbol	str
degree	int
component_id	int
affected	bool, initially false

Node keys are canonical atom indices.

Node positions are the selected frame's Cartesian coordinates for active atoms.

Atomic edge attributes

The adapter must provide at least:

atom_i	int
atom_j	int
source_symbol	str
target_symbol	str
species_pair	tuple[str, str]
canonical_image_shift	tuple[int, int, int]
display_image_shift	tuple[int, int, int]
periodic	bool
transition_status	"unchanged"

For a state view, edge keys are the authoritative canonical AtomicEdgeKey objects. The view's edge_image_shifts are display shifts, while the canonical shifts remain in canonical_image_shift.

The adapter metadata must include:

- adapter schema version;
- selected collection frame index and frame ID;
- source state digest;
- source connectivity-definition kind when available;
- source result consistency when available;
- a statement that display shifts were reconstructed from canonical topology and selected-frame geometry.

Canonical versus frame-consistent image shifts

The problem

AtomicConnectivityState.edge_image_shifts are canonical graph-identity data. They have been normalized by a periodic vertex gauge. Selected frame coordinates may use a different wrapping gauge.

Drawing canonical shifts directly against those wrapped coordinates can produce incorrect edge geometry.

Let $\mathbf{m}_{ij}^c$ be the canonical shift, $\mathbf{m}_{ij}^f$ the shift consistent with the selected frame, and $\mathbf{g}_i$ an integer vertex gauge. The atomic module uses

$$\mathbf{m}_{ij}^c = \mathbf{m}_{ij}^f + \mathbf{g}_i - \mathbf{g}_j.$$

Therefore

$$\mathbf{m}_{ij}^f = \mathbf{m}_{ij}^c - \mathbf{g}_i + \mathbf{g}_j.$$

Reconstruction algorithm

The private adapter helper must:

1. construct a deterministic spanning forest of the canonical state;
2. choose the smallest atom index as the root of each component;
3. set the root gauge to zero;
4. for each oriented tree edge $i \rightarrow j$, compute the selected frame's minimum-image shift μ_{ij} ;
5. propagate

$$\mathbf{g}_j = \mathbf{g}_i + \mu_{ij} - \mathbf{m}_{ij}^c;$$

6. reconstruct every edge with

$$\mathbf{m}_{ij}^f = \mathbf{m}_{ij}^c - \mathbf{g}_i + \mathbf{g}_j;$$

7. verify zero shifts on nonperiodic axes;

8. verify finite reconstructed Cartesian vectors;
9. preserve non-tree winding information from the canonical graph.

This is geometry reconstruction, not connectivity analysis. The adapter must not reapply cutoffs or decide whether an edge exists.

Minimum-image ties should produce a warning because the display gauge may be ambiguous, even though graph identity remains valid.

Atomic transition adapter

Public definition

```
def graph_view_from_connectivity_transition(
    collection: AtomisticFrameCollection,
    result: AtomicConnectivityResult,
    *,
    transition_id: int,
    coordinate_frame: Literal["before", "after"] = "after",
) -> DecoratedGraphView:
    ...
```

The transition ID must exist. The result must have trajectory semantics and contain transition records.

Transition graph construction

Let the source and target edge-pair sets be E_A and E_B . The display graph uses

$$E_{\text{display}} = E_A \cup E_B.$$

Each pair receives one status:

$$\text{status}(e) = \begin{cases} \text{removed}, & e \in E_A \setminus E_B, \\ \text{added}, & e \in E_B \setminus E_A, \\ \text{unchanged}, & e \in E_A \cap E_B. \end{cases}$$

Matching uses unordered atom-pair identity, consistent with the atomic-connectivity transition contract.

The selected coordinate frame supplies all node positions. Source and target graph shifts are reconstructed separately against that frame. Unchanged edges use the state corresponding to the selected coordinate frame. Removed edges use source canonical topology; added edges use target canonical topology.

The union view is a diagnostic display graph, not a new atomic-connectivity state.

Transition keys and attributes

Transition edge keys should be stable tuples:

```
(status, atom_i, atom_j)
```

The edge attributes must include:

```
transition_status
source_canonical_image_shift or None
target_canonical_image_shift or None
display_image_shift
atom_i
atom_j
species_pair
periodic
```

Node attribute affected is true exactly for atoms listed in the scientific transition record.

Atomic convenience plotting functions

```
def plot_atomic_connectivity_2d(
    collection: AtomisticFrameCollection,
    connectivity: AtomicConnectivityState |
        AtomicConnectivityResult,
    *,
    frame_index: int,
    layout: GraphLayoutOptions | None = None,
    style: GraphStyle | None = None,
    focus: GraphFocus | None = None,
    graph_filter: GraphFilter | None = None,
    complexity_policy: GraphComplexityPolicy | None = None,
    periodic: PeriodicDisplayOptions | None = None,
    options: Graph2DRenderOptions | None = None,
    axes: matplotlib.axes.Axes | None = None,
) -> GraphRenderResult:
    ...
```

The default style is `GraphStyle.atomic_default()`.

```
def plot_connectivity_transition_2d(
    collection: AtomisticFrameCollection,
    result: AtomicConnectivityResult,
    *,
    transition_id: int,
    coordinate_frame: Literal["before", "after"] = "after",
    layout: GraphLayoutOptions | None = None,
    style: GraphStyle | None = None,
    focus: GraphFocus | None = None,
    graph_filter: GraphFilter | None = None,
    complexity_policy: GraphComplexityPolicy | None = None,
    periodic: PeriodicDisplayOptions | None = None,
    options: Graph2DRenderOptions | None = None,
    axes: matplotlib.axes.Axes | None = None,
) -> GraphRenderResult:
    ...
```

The default style is `GraphStyle.transition_default()`.

Both wrappers must construct a view and delegate to `plot_decorated_graph_2d()`. They must not duplicate layout, filtering, or artist logic.

Real-structure system-integration fixture

The release includes the user-supplied relaxed Na-LTA POSCAR as `tests/data/Na_LTA_relaxed.POSCAR`. The authoritative framework diagnostic uses the first 144 Si/Al/O atoms and the explicit cutoffs

$$r_{\text{Si-O}} < 2.0 \text{ \AA}, \quad r_{\text{Al-O}} < 2.0 \text{ \AA} .$$

The resulting graph contains 144 active atoms and 192 T-O edges. Every Si/Al site has degree four and every framework oxygen has degree two. This test simultaneously checks structure reading, connectivity construction, periodic display-shift reconstruction, graph adaptation, physical projection, and rendering.

A separate gallery view includes Na-O contacts using an illustrative 3.15 Å cutoff. That value is a visualization fixture, not a universal chemical definition and not part of the authoritative framework-connectivity test.

Adapter metadata schemas

The state adapter records schema identifier `mdstats.atomic-connectivity-graph-view.v1`. The transition adapter records `mdstats.connectivity-transition-graph-view.v1`.

State node attributes include atom index, atomic number, symbol, degree, component, and an affected flag initialized to false. State edge attributes include endpoint atom indices, endpoint symbols, species pair, canonical image shift, display image shift, periodic status, and transition status.

Transition nodes retain the common atom population and mark affected atoms. Transition edges form the union of source and target atom pairs and carry source/target canonical shifts, a coordinate-frame display shift, endpoint metadata, periodic status, and a transition class.

Input constraints

The selected frame must exist in the collection. Active atom indices and atomic numbers must match the collection's canonical atom ordering. PBC flags must agree. The selected frame cell must be finite and nonsingular, and wrapped coordinates must be finite.

For a transition view, source and target states must use the same active atom identities. The requested transition ID must be present exactly once. The coordinate frame must be either before or after.

Error and warning policy

Inconsistent scientific inputs raise `GraphAdapterError`. Type errors are raised when the public function receives an unsupported result type. Minimum-image ties during display-shift reconstruction are retained as adapter warnings in view metadata rather than silently ignored.

Edge cases and cautions

- A broad connectivity scope may include mobile-ion contacts that obscure the rigid framework; this is a scientific scope choice, not a renderer defect.
- A two-dimensional physical projection may place unrelated atoms on top of one another.
- Canonical image shifts cannot generally be combined directly with independently wrapped coordinates.
- Local unwrapping cannot eliminate all winding in a periodic graph containing noncontractible cycles.
- Transition edge matching currently uses unordered atom-pair identity, consistent with the first atomic-connectivity release.
- The illustrative Na-O cutoff in the integration gallery is not a universal bond definition.

Testing requirements

Tests must cover:

- state/result frame lookup;
- collection/state identity validation;
- node and edge metadata values and lengths;
- frame-local display-shift reconstruction;
- invariance under atom wrapping gauge;
- non-tree periodic winding preservation;
- minimum-image tie diagnostics;
- transition union classification;
- before/after coordinate-frame selection;
- default atomic and transition wrapper styles;
- end-to-end rendering with the Na-LTA fixture.

The Na-LTA framework acceptance check is:

$$N_T = 48, \quad N_O = 96, \quad N_E = 192,$$

with degree four on every Si/Al site and degree two on every framework oxygen.

Public exports

```
graph_view_from_atomic_connectivity
graph_view_from_connectivity_transition
plot_atomic_connectivity_2d
```

```
plot_connectivity_transition_2d
plot_atomic_connectivity_3d
plot_connectivity_transition_3d
```

Future compatibility

The 3-D wrappers reuse the same state and transition adapters rather than creating a second scientific conversion path. Framework and ring adapters remain separate modules because they consume different authoritative scientific graph objects and metadata.

Interactive 3-D wrappers

The state and transition adapters remain the only scientific conversion path for atomic connectivity. Stage G5 provides convenience wrappers that reuse the existing view constructors and delegate to the generic Plotly renderer.

```
def plot_atomic_connectivity_3d(
    collection: AtomisticFrameCollection,
    connectivity: AtomicConnectivityState | AtomicConnectivityResult,
    *,
    frame_index: int,
    periodic: PeriodicDisplayOptions | None = None,
    style: GraphStyle | None = None,
    focus: GraphFocus | None = None,
    graph_filter: GraphFilter | None = None,
    complexity_policy: GraphComplexityPolicy | None = None,
    options: Graph3DRenderOptions | None = None,
) -> InteractiveGraphRenderResult:
    ...

def plot_connectivity_transition_3d(
    collection: AtomisticFrameCollection,
    result: AtomicConnectivityResult,
    *,
    transition_id: int,
    coordinate_frame: Literal["before", "after"] = "after",
    periodic: PeriodicDisplayOptions | None = None,
    style: GraphStyle | None = None,
    focus: GraphFocus | None = None,
    graph_filter: GraphFilter | None = None,
    complexity_policy: GraphComplexityPolicy | None = None,
    options: Graph3DRenderOptions | None = None,
) -> InteractiveGraphRenderResult:
    ...
```

The wrappers must:

1. construct the same DecoratedGraphView used by the 2-D wrappers;
2. use GraphStyle.atomic_default() or GraphStyle.transition_default() when no style is supplied;
3. pass focus, filtering, periodic display, complexity, and renderer options through unchanged;
4. return the generic InteractiveGraphRenderResult;
5. perform no duplicate display-shift reconstruction, periodic replication, or Plotly trace construction.

The G5 public exports are:

```
plot_atomic_connectivity_3d
plot_connectivity_transition_3d
```

These additions do not change state or transition adapter metadata schemas.